Trabalho Prático n.º 1
UC - Métodos Numéricos

202405814 Guilherme Rodrigues
202406361 Lara Dinis
202405276 Miguel Ribeiro

Outubro 2025

Introdução

No âmbito do desenvolvimento e resolução deste trabalho, decidimos utlilizar o C como lingua de programação, em virtude desta ser uma linguagem compilada, o que proporciona uma execução mais rápida e eficiente, o qual considerámos crucial para lidar com a grande dimensão dos cálculos numéricos exigidos.

Exercício 1

O objetivo deste exercício é determinar o valor do *epsilon da máquina*, isto é, o menor número positivo que, quando adicionado a 1, produz um valor diferente de 1 na representação interna do computador.

Em C, dispomos de três tipos de dados capazes de representar números em vígula flutuante: `float`, `double` e `long double`. Em cada caso, o algoritmo divide sucessivamente o valor inicial de *epsilon* pela metade até que a condição "(1 + epsilon/2) == 1" deixe de ser satisfeita, ponto no qual encontrámos o *epsilon da máquina*.

Tendo em conta os resultados obtidos, optámos por trabalhar com o tipo `double` nos exercicios seguintes, pois este oferece um bom equilíbrio entre **precisão** e **desempenho**.

```
Epsilon para float: 1.1920928955078125000e-07
Epsilon para double: 2.2204460492503130808e-16
Epsilon para longdouble: 1.0842021724855044340e-19
```

Figure 1: Epsilon da máquina para os tipos de dado`float`, `double` e `long double`

Exercício 2

Pretende-se aproximar o valor da série

$$S \;=\; 6\sum_{k=0}^{\infty} \frac{(2k)!}{4^k\,(k!)^2\,(2k+1)}\,(0.5)^{2k+1}\,.$$

Seja a_k uma sucessão tal que:

$$a_k \;=\; \frac{(2k)!}{4^k\,(k!)^2\,(2k+1)}\,(0.5)^{2k+1} \quad \text{e assim,} \qquad S = 6\sum_{k=0}^{\infty} a_k.$$

Pretendemos então, para um dado ε, determinar qual o termo n da sucessão, tal que a soma parcial $\sum_{k=0}^{n} a_k$ aproxime o resultado com erro absoluto inferior a ε.

2.1. Série de Termos Positivos

Visto esta sucessão tratar-se de uma série de termos positivos, procedemos a averiguar se a mesma é convergente e, caso isso se verifique, qual o valor absoluto do erro de truncatura R_n (erro absoluto com o qual a soma parcial S_n aproxima S) para o qual:

$$R_n \;\leq\; a_{n+1}\,\frac{1}{1-L}.$$

Determinemos o valor de $L = \lim_{n\to\infty} \frac{a_{n+1}}{a_n}$:

$$L = \lim_{n\to\infty} \frac{(2(n+1))!(0.5)^{2(n+1)+1}}{4^{n+1}((n+1)!)^2(2(n+1)+1)} \cdot \frac{4^n(n!)^2(2n+1)}{(2n)!(0.5)^{2n+1}} =$$

$$= \frac{1}{16}\lim_{n\to\infty} \frac{(2n+2)(2n+1)^2}{(2n+3)(n+1)^2} = \frac{1}{8}\lim_{n\to\infty} \frac{(2n+1)^2}{(2n+3)(n+1)} = \frac{1}{8}\lim_{n\to\infty} \frac{4n^2}{2n^2} = \frac{1}{8}\cdot 2 = \frac{1}{4}.$$

Sabendo que $L = \frac{1}{4}$, podemos concluir que a sucessão é convergente ($L < 1$, pelo Critério de D'Alembert) e o erro é majorado por:

$$R_n \leq 6 \cdot \frac{4}{3}\,a_{n+1} = 8a_{n+1}.$$

2.2. Fórmula Recursiva

Calcular a_n diretamente pela fórmula dada exige o cálculo de valores maiores que o representável pela máquina, sendo propenso a overflows. A transformação da sucessão numa relação recursiva permite obter cada novo termo com custo mínimo, a partir do anterior. Temos então:

$$\frac{a_{n+1}}{a_n} = \frac{(2n+1)^2}{8(2n+3)(n+1)} \iff a_{n+1} = a_n \cdot \frac{(2n+1)^2}{8(2n+3)(n+1)}, a_0 = 0.5.$$

2.3. Implementação em C

Após obtermos a expressão a_n na sua forma recursiva, e sabendo como majorar o erro, já somos capazes de implementar o restante cálculo em C.

O programa calcula uma aproximação ao valor de S com erro absoluto inferior ao ε dado, do seguinte modo:

1. Inicializa o primeiro termo da série, $a_0 = 0.5$, somando-o a S.
2. Utiliza a fórmula recursiva calculada para gerar o termo seguinte.
3. Após cada iteração, verifica o critério de paragem $8a_{n+1} \leq \varepsilon$, que garante que o erro absoluto é inferior ao valor pedido.
4. Se a condição se verificar, adiciona o termo atual à soma e atualiza os contadores.
5. Quando a condição já não for satisfeita, o ciclo é interrompido, sendo devolvidos a soma da série até ao maior n que satisfaz a condição e o valor desse n.

2.4. Resultados

```
eps = 1e-05 -> n = 7 termos, S = 3.1415894253191219
eps = 1e-10 -> n = 14 termos, S = 3.1415926535153385
eps = 1e-15 -> n = 22 termos, S = 3.1415926535897936
```

Figure 2: Aproximação de S com erro absoluto majorado inferior a ε

Observa-se que, dado o reduzido número de termos necessários para a interrupção do programa, esta série converge para um valor próximo de π rapidamente, revelando-se extremamente prática e exigindo reduzido esforço computacional. Verifica-se também, como expectável, que o aumento da precisão em função da minimização do ε aproxima a série de forma segura de π, precisando apenas de 22 iterações para uma representação precisa, com 16 algarismos significativos.

Exercício 3

No exercício 3, pretendemos aproximar o valor da série,

$$S \;=\; 4\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)}.$$

Seja b_k uma sucessão tal que:

$$b_k = \frac{(-1)^k}{(2k+1)} \quad \text{e} \quad S = 4\sum_{k=0}^{\infty} b_k.$$

3.1. Série de Termos Positivos e Negativos

Neste caso, encontramo-nos perante uma série de termos positivos e negativos, o que nos obriga a adequar o nosso método para lidar com este tipo de série. Procedemos a determinar a sua convergência, verificando que $|b_{n+1}| < |b_n|$ e $\lim_{n\to\infty} b_n = 0$ e, caso estas condições se verifiquem, qual o valor absoluto do erro de truncatura R_n para o qual:

$$|R_n| < |b_{n+1}|$$

Assim:

$$\left|\frac{b_{n+1}}{b_n}\right| = \left|\frac{(-1)^{n+1}}{2(n+1)+1} \cdot \frac{2n+1}{(-1)^n}\right| = \left|\frac{-(2n+1)}{2n+3}\right| < 1,$$

logo, verifica-se que $|b_{n+1}| < |b_n|$, $\forall n \in \mathbf{N}$. Relativamente a $\lim_{n\to\infty} b_n$:

$$\lim_{n\to\infty} \frac{(-1)^n}{(2n+1)} = \lim_{n\to\infty} (-1)^n \cdot \lim_{n\to\infty} \frac{1}{2n+1} = 0 \cdot \lim_{n\to\infty} (-1)^n = 0.$$

Tendo verificado ambas as condições, concluimos que a série converge, sendo o erro majorado por $|R_n| < 4 \cdot |b_{n+1}|$.

3.2. Implementação em C

Ao contrário do exercício 2, decidimos não optar por uma abordagem recursiva, dada a simplicidade da sua iteração, uma vez que a expressão não envolve operações complexas, exceto pela determinação do sinal do termo para valores grandes de k. Assim, o programa calcula uma aproximação ao valor de S com erro absoluto inferior ao ε dado, do seguinte modo:

1. Inicializa a variável `soma` ($= 0$), que irá armazenar o valor acumulado da série.

2. A cada iteração, é calculado o sinal do termo através de uma função que determina a paridade de k.

3. Para cada termo, é verificado o critério de paragem, $4 \cdot |b_{n+1}| \leq \varepsilon$, que garante que o erro absoluto é inferior ao ε pedido.

4. Se a condição não se verificar, o termo é acrescentado à soma, e o contador é incrementado, repetindo o processo para o termo seguinte.

5. Quando o critério de paragem é satisfeito, o ciclo é interrompido, sendo devolvidos o número total de termos somados e o valor aproximado da série.

3.4. Resultados

```
eps = 1e-05 -> n = 200000 termos, S = 3.1415876535897724
eps = 1e-10 -> n = 20000000012 termos, S = 3.1415926535384981
```

Figure 3: Aproximação de S com erro absoluto majorado inferior a ε

Observa-se que, à medida que o valor de ε diminui, o número de termos necessários para atingir a precisão pretendida aumenta de forma significativa.

Embora o programa implementado seja eficiente e simples, a série em questão converge muito lentamente para π, pelo que, para valores de ε muito pequenos (como 10^{-15}), o número de termos exigido para atingir o erro pretendido é simplesmente enorme, tornando o seu cálculo inviável em tempo útil.

Assim, apenas são apresentados os resultados para $\varepsilon = 10^{-5}$ e $\varepsilon = 10^{-10}$, que já demonstram claramente o padrão de convergência lenta da série e a sua aproximação gradual a π.

3.5. Aceleração da Convergência (Transformação de Euler)

Como visto na análise anterior, a série converge muito lentamente para o valor de π, sendo a sua execução inviável para valores de ε muito pequenos.

Para contornar esta situação, aplicaremos a **Transformação de Euler** para séries alternadas, que reescreve S numa nova série de termos positivos e decrescentes:

$$\sum_{n=0}^{\infty} c_n, \qquad c_n = 2^{-(n+1)} \sum_{k=0}^{n} \binom{n}{k} b_k,$$

que converge para o mesmo resultado, mais rapidamente, e cujo erro de truncatura é majorado por:

$$|R_n| < 4 \cdot c_{n+1}.$$

3.5.1. Implementação em C com Aceleração da Convergência

O programa calcula uma aproximação ao valor de S com erro absoluto inferior a ε, aplicando a Transformação de Euler, do seguinte modo:

1. Definem-se as variáveis `soma` ($= 0$), e `n` ($= 0$) como índice do termo acelerado c_n.
2. A variável `termo euler` ($= 0$) armazena o valor da série composta pelo produto entre o coeficiente binomial e o valor de b_k, que é dividido por $2n + 1$ vezes e acrescentado à soma da nova série.
3. A cada iteração, é verificada a condição de paragem, $4 \cdot c_{n+1} \leq \varepsilon$.

4. Caso não seja satisfeita, o valor de n é incrementado, repetindo-se todo o processo.

5. Quando a condição é satisfeita, é devolvida o valor da aproximação de S, assim como o número de termos necessários à sua concretização.

3.5.2. Resultados da Aceleração da Convergência

```
eps = 1e-05 -> n = 17 termos, S = 3.1415797881375949
eps = 1e-10 -> n = 33 termos, S = 3.1415926534476353
eps = 1e-15 -> n = 49 termos, S = 3.1415926535897913
```

Figure 4: Aproximação de S com erro absoluto majorado inferior a ε com aceleração da convergência (Transformação de Euler)

Observa-se que, com a aplicação da Transformação de Euler, o número de termos necessários para atingir uma dada precisão diminui significativamente. Enquanto a série original requer um número extremamente elevado de termos para convergir, esta permite obter aproximações igualmente precisas de forma mais eficiente. Por exemplo, para $\varepsilon = 10^{-5}$ bastam apenas 17 termos, para $\varepsilon = 10^{-10}$ apenas 33 termos, e para $\varepsilon = 10^{-15}$ 49 termos são suficientes (conforme mostrado na Figura 4), números muito mais favoráveis do que os valores exorbitantes obtidos anteriormente.

Concluímos, então, que este método reduz drasticamente o número de termos necessários para atingir valores igualmente precisos, tornando viável a aproximação de π com ε muito pequenos e num tempo de execução ótimo.

Exercício 4

Sabendo que $S = \pi$, para ambas as séries, procedemos ao cálculo do erro absoluto efetivamente cometido nos exercícios 2. e 3., comparando-o com os valores de erro absoluto fornecidos em cada caso (10^{-5}, 10^{-10} e 10^{-15}), que é dado por:

$$E \quad = \quad |\pi - S|.$$

Em C, facilmente implementamos este cálculo, obtendo os seguintes resultados para o erro absoluto cometido:

```
eps = 1e-05 -> n = 7 termos, S = 3.1415894253191219, erro = 3.23e-06
eps = 1e-10 -> n = 14 termos, S = 3.1415926535153385, erro = 7.45e-11
eps = 1e-15 -> n = 22 termos, S = 3.1415926535897936, erro = 4.44e-16
```

Figure 5: Erro absoluto efetivamente cometido no ex.2 no cálculo de π

```
eps = 1e-05 -> n = 200000 termos, S = 3.1415876535897724, erro = 5.00e-06
eps = 1e-10 -> n = 20000000012 termos, S = 3.1415926535384981, erro = 5.13e-11
```

Figure 6: Erro absoluto efetivamente cometido no ex.3 no cálculo de π

```
eps = 1e-05 -> n = 17 termos, S = 3.1415797881375949, erro = 1.29e-05
eps = 1e-10 -> n = 33 termos, S = 3.1415926534476353, erro = 1.42e-10
eps = 1e-15 -> n = 49 termos, S = 3.1415926535897913, erro = 1.78e-15
```

Figure 7: Erro absoluto efetivamente cometido no ex.3 no cálculo de π com aceleração da convergência (Transformação de Euler)

Através da análise dos resultados obtidos, podemos concluir que, em relação aos Exercícios 2 e 3 (Figuras 5 e 6), verifica-se que em ambos os casos o erro absoluto efetivo foi sempre inferior ao erro máximo pretendido, confirmando a validade dos majorantes de erro utilizados.

À medida que o valor de ϵ diminui, ambas as séries aproximam-se progressivamente do valor real de π, embora com eficiências bastante distintas, enquanto a série do Exercício 2 atinge precisão de 10^{-15} com apenas 22 termos, a série de Leibniz do Exercício 3 requer um número substancialmente maior de termos para a mesma precisão.

Quanto à Transformação de Euler aplicada no Exercício 3 (Figura 7), observa-se uma melhoria significativa face à série original. Esta técnica de aceleração de convergência demonstrou ser bastante eficaz, reduzindo drasticamente o número de termos necessários para atingir cada nível de precisão.

Comparando com a Figura 6, a transformação de Euler consegue atingir erros da ordem de 10^{-15} com apenas 49 termos, mostrando como esta técnica consegue acelerar consideravelmente a convergência da série.